

CONTAINER



ESEGUI IL TUO PRIMO CONTENITORE

Per eseguire un contenitore con Docker, puoi usare un semplice comando:

```
docker run -it --rm debian:bullseye
```

Questo comando crea un nuovo contenitore interattivo (flag `-it`) utilizzando l'immagine `debian:bullseye` e rimuove il contenitore all'uscita (flag `--rm`).



I. COMPILARE SOFTWARE È FASTIDIOSO

- **Scenario:** Tenti di compilare e ottieni errori come `ERRORE`: hai la versione 2.1.1 e ti serve almeno la 2.2.4.



2. DISTRIBUIRE IL SOFTWARE È FASTIDIOSO

- **Scenario:** "ugh, il mio sito web è rotto perché ho usato una funzionalità di Python 3.6 e il server ha solo Python 3.5".



BIG BRAIN MOMENT: INCLUDERE OGNI DIPENDENZA



COSA C'È IN UN'IMMAGINE CONTAINER?

Un'immagine container è essenzialmente un **archivio tarball di un filesystem**.

Costruisce un ambiente isolato e portatile.

Per un'applicazione **Python** (ad esempio, un'applicazione web Django o Flask), i contenuti tipici che compongono l'immagine container potrebbero includere:

- **Il codice della tua applicazione Python** (script, moduli, file `settings.py`, ecc.)
- **Dipendenze del runtime del sistema:** Librerie essenziali del sistema operativo (come `libc`) e altre utility di base.
- **Sistema operativo di base:** Una distribuzione Linux minimale (es. Alpine, Debian Slim o Ubuntu), che fornisce il contesto per l'esecuzione.
- **Interprete Python:** La versione specifica di Python (es. Python 3.11) necessaria per eseguire l'applicazione.

- ☐ • **Pacchetti Python (Dependencies):** Tutte le librerie e i framework Python specifici dell'applicazione (es. Django, Flask, Pandas, requests), installati

tramite `pip` e specificati nel file `requirements.txt`.

COME VENGONO COSTRUITE LE IMMAGINI

1. Parti da un sistema operativo di base.
2. Installa il programma + le dipendenze.
3. Configuralo come desideri.
4. Crea un tarball dell'**INTERO FILESYSTEM**.
 - *Questo è ciò che fa `docker build`!*



ESEGUIRE UN'IMMAGINE

1. Scarica il tarball.
2. Decomprimi in una directory.
3. Esegui un programma e fai finta che quella directory sia il suo intero filesystem.

Vantaggio: Le immagini ti permettono di "installare" programmi molto facilmente (ad esempio, configurare un database di test Postgres in pochi secondi).



I CONTAINER NON SONO MAGICI

Queste 15 righe di bash avvieranno un container che esegue la shell fish.

Funziona solo su Linux perché queste funzionalità sono tutte esclusive di Linux.

```
wget bit.ly/fish-container -O fish.tar           # 1. Scarica l'immagine
mkdir container-root; cd container-root          # 2. Prepara la directory
tar -xf ../fish.tar                             # 2. Decomprimi l'immagine

cgroup_id="cgroup_$(shuf -i 1000-2000 -n 1)"    # 3. Genera un nome per il cgroup
cgcreate -g "cpu,cpuacct,memory:$cgroup_id"     # 4. Crea il cgroup
cgset -r cpu.shares=512 "$cgroup_id"            # 4. Imposta i limiti di CPU
cgset -r memory.limit_in_bytes=100000000 "$cgroup_id" # 4. Imposta i limiti di
Memoria

cgexec -g "cpu,cpuacct,memory:$cgroup_id" \    # 5. Usa il cgroup
unshare -fmupn --mount-proc \                  # 6. Crea e usa i namespaces
chroot "$PWD" \                                # 7. Cambia la directory root
/bin/sh -c "                                    # 10. Avvia la shell
    /bin/mount -t proc proc /proc &&           # 8. Usa il /proc corretto
    hostname container-fun-times &&            # 9. Cambia l'hostname
    /usr/bin/fish                             # 10. Avvia fish!
"
```

CONTAINER = PROCESSI

- Un container è un gruppo di processi Linux.
- I processi del container possono fare tutto ciò che può fare un normale processo.
- **Nota:** Su un Windows, tutti i tuoi container sono in realtà in esecuzione in una macchina virtuale Linux.



DENTRO VS FUORI

Dentro il container:

```
$ ps aux | grep top
USER PID      START COMMAND
root  1        20:55 top
```

Fuori dal container (Host):

```
$ ps aux | grep top
USER PID      START COMMAND
root 23540 20:55 top
hide 23546 20:57 top    #è sempre lui!
```

Intuizione chiave: Questi due sono lo **stesso processo**! Le restrizioni sono imposte dal kernel Linux (namespace PID diverso, memoria limitata tramite cgroups, directory root diversa, capability limitate).

"Container" non ha una definizione chiara, ma i container Docker usano tutte queste funzionalità del kernel Linux:

1. **pivot_root**: Imposta la directory root di un processo su una directory con il contenuto dell'immagine container.
2. **cgroups**: Limita l'uso di memoria/CPU per un gruppo di processi
3. **namespaces**: Permette ai processi di avere i propri:
 - rete
 - PID
 - hostname
 - mount
 - utenti
4. **capabilities**: Sicurezza: fornisce permessi specifici.
5. **seccomp-bpf**: Sicurezza: previene chiamate di sistema pericolose.
- ☐ 6. **overlay filesystems**: Questo è ciò che fa funzionare i **layer**! La condivisione dei layer fa risparmiare spazio su disco e aiuta i container ad avviarsi più

velocemente.

CHROOT VS PIVOT_ROOT

chroot: Cambia la directory root di un processo.

- Se fai chroot in /fake/root, quando apre /usr/bin/redis otterrà invece /fake/root/usr/bin/redis.
- **Problema:** I programmi possono uscire da un chroot. Tutti i vecchi file sono ancora lì e un processo root può accedervi se lo desidera.

pivot_root:

- I container usano pivot_root invece di chroot.
- Puoi smontare il vecchio filesystem in modo che sia impossibile accedervi.

Nota: Per avere un "container" ti serve più di pivot_root. Da solo, non ti permetterà di impostare limiti di CPU/memoria, nascondere altri processi, usare la stessa porta di un altro processo o limitare chiamate di sistema pericolose.



LAYER

Immagini diverse hanno spesso file simili.

- **Immagine container Rails:** Usa Ubuntu 18.04
- **Immagine container Django:** Usa Ubuntu 18.04

Il riutilizzo dei layer fa risparmiare spazio su disco.

- File esattamente identici su disco per il layer del sistema operativo di base!



COME FUNZIONANO I LAYER

- **ID del Layer:** Ogni layer ha un ID, di solito un hash sha256 del contenuto del layer (es. 8e99fae2. .).
- **Struttura:** Un layer è solo una directory contenente file (es. /bin, /lib, /etc).
- **Unione:** Se un file si trova in 2 layer, vedrai la versione del **layer superiore**.

Layer di Scrittura (Write Layer): Per impostazione predefinita, le scritture vanno su un layer temporaneo. Questi file potrebbero essere eliminati dopo l'uscita dal container. Per mantenere le modifiche, scrivi su una directory montata dall'esterno del container.



FILESYSTEM OVERLAY

Come funziona l'unione dei layer?

Se hai due o più directory, come le fai sembrare una sola?

Soluzione di Linux: I filesystem Overlay (come OverlayFS).

- **Layer superiore (Writable Layer):** Tutte le modifiche vengono scritte qui. Se modifichi un file che esiste già in un layer inferiore, viene copiato nel Writable Layer e modificato lì.
- **Layer inferiore (Base Layers):** Sono tutti in sola lettura (read-only).

Vantaggio: Condivisione dei file. Non appena ottieni un layer base, se un'altra immagine utilizza lo stesso layer base, non devi scaricarlo di nuovo. Questo è un enorme risparmio di spazio su disco.



CONTAINER REGISTRIES

- **Definizione:** Un Container Registry (come Docker Hub) è un server per archiviare e distribuire immagini.
- **Funzionamento:**
 1. Esegui `docker push` su un'immagine (un set di layer).
 2. Il client Docker invia ciascun layer al Registry.
 3. Il Registry archivia i layer.
- **Pull dell'Immagine:** Quando esegui `docker pull`, il client controlla quali layer hai già e scarica solo quelli mancanti.
- **Manifest:** Un file chiamato "manifest" dice al Registry quali layer compongono l'immagine, l'ordine e il checksum.



CGROUPS

cgroups (Control Groups): La funzionalità del kernel Linux per **limitare le risorse** che un processo può utilizzare.

- **Senza cgroups:** Un processo malfunzionante o dannoso può saturare la CPU o esaurire la memoria, bloccando l'intero sistema.
- **Con cgroups:** Puoi dire al kernel che un determinato gruppo di processi può utilizzare, per es., **al massimo** 500 MB di RAM e il 10% della CPU.



Funzionamento:

1. **Crea un cgroup:** Usa un comando come `cgcreate -g cpu,memory:my-container-1`.
 2. **Imposta i limiti:** Ad esempio, `cgset -r memory.limit_in_bytes=524288000 my-container-1`.
 3. **Esegui il processo:** Usa `cgexec -g cpu,memory:my-container-1 /usr/bin/redis-server`. Il server Redis sarà ora limitato dalle risorse che hai specificato.
- **Nota:** I cgroups sono un meccanismo di **isolamento delle risorse**, non di isolamento del filesystem o di rete (per questo servono i namespaces).



NAMESPACES

I **Namespaces** sono la funzionalità del kernel Linux per l'**isolamento** tra processi.

- **Problema:** Se avvii un processo su Linux, di default vede **tutti** gli altri processi, **tutte** le schede di rete, **tutti** i filesystem montati.
- **Soluzione:** I Namespaces ingannano i processi, facendogli credere di essere l'unico processo/rete/utente sul sistema.



TIPI DI NAMESPACE

Namespace	Isolamento	Cosa rende unico
PID	Processi	La sua tabella dei processi
Net	Rete	Interfacce di rete, indirizzo IP, porte
User	Utente	ID utente e ID gruppo
Mount	Filesystem	Punti di mount
UTS	Nome host	Nome host
IPC	Comunicazione interprocesso	Code di messaggi, semafori



COME CREARE UN NAMESPACE

Un processo si unisce o crea un namespace per ogni tipo. Ad esempio, il processo del tuo server web è probabilmente nel namespace PID, Net, User, Mount, UTS, IPC di default del sistema host.



CREAZIONE DI UN NAMESPACE CON UNSHARE

Il comando unshare esegue un programma in un nuovo namespace.

Esempio:

```
# Esegue 'sh' nel proprio namespace per il nome host (UTS)
$ sudo unshare -u sh
# Ora esegui 'hostname'
$ hostname
# Output: [Il nome host del sistema host]
# Cambia il nome host:
$ hostname my-secret-container
# Ora esegui 'hostname' di nuovo
$ hostname
# Output: my-secret-container
# Se apri un'altra shell 0 sul sistema host, il nome host non è cambiato.
# L'effetto è locale a questa shell (perché è nel suo namespace UTS).
```



PID NAMESPACES

PID Namespaces: Danno a un processo la propria tabella PID (Process ID).

Se avvii il server Redis (PID 1234) e poi `top` (PID 5678) in un contenitore:

- All'interno del contenitore, i processi pensano che il server Redis sia PID 1.
- Fuori dal contenitore (Host), vedono ancora i PID reali (1234 e 5678).
- All'interno del contenitore, i processi non possono vedere (e quindi non possono uccidere) i processi al di fuori del loro namespace.
- Il processo principale del contenitore è sempre PID 1, semplificando la scrittura di script di avvio.



NETWORK NAMESPACES

Network Namespaces: Forniscono a un processo la propria configurazione di rete (interfacce, indirizzi IP, tabella di routing, regole del firewall).

Componenti di un Net Namespace:

- Interfaccia 1o (loopback)
- **Interfaccia virtuale veth:**
 - Un'estremità va nel Namespace del Container.
 - L'altra estremità va nel Namespace dell'Host (di solito collegata a un bridge).

Vantaggio: Puoi eseguire decine di contenitori sulla porta 80, ognuno con il proprio indirizzo IP interno.



Contenitori docker: Indirizzi IP dei Container

1. **Bridge:** Il software del container (Docker) crea un bridge di rete (come `docker0` su Docker).
2. **veth Pair:** Viene creata una coppia di interfacce virtuali (`veth`).
Un'estremità viene inserita nel Net Namespace del container e l'altra viene collegata al bridge `docker0`.
3. **Indirizzo IP:** Il container ottiene un indirizzo IP (ad esempio `172.17.0.2`) dal pool del bridge.
4. **Connessione a Internet:** Il bridge `docker0` utilizza il NAT (Network Address Translation) per instradare il traffico dei container verso l'interfaccia di rete reale dell'Host (ad esempio, `eth0`).



Contenitori docker: Esporre una porta

Quando esegui `docker run -p 8080:80`, Docker fa 2 cose:

1. Avvia il container sulla sua porta 80 virtuale.
2. Aggiunge una regola di firewall (iptables) all'Host che reindirizza il traffico in arrivo sulla porta 8080 dell'Host alla porta 80 del container.



CAPABILITIES

Capabilities: Sono granularità (specifici permessi) per la sicurezza di Linux.

- **Problema:** Storicamente, su Linux c'erano solo due tipi di utenti: root (ha tutti i permessi) e *non-root* (non ha permessi speciali).
- **Soluzione:** Le Capabilities suddividono i permessi di root in centinaia di funzioni più piccole.

Esempio di Capabilities:

- **CAP_NET_BIND_SERVICE:** Permette di eseguire il bind a porte sotto 1024.
- **CAP_SETUID:** Permette di cambiare l'ID utente.
- **CAP_SYS_ADMIN:** Permette di eseguire operazioni di amministrazione di sistema (la più pericolosa).



I contenitori di Docker (di default):

- **Rimuovono** molte delle capabilities di root (ad esempio, CAP_SYS_ADMIN viene rimosso per impostazione predefinita).
- **Mantengono** un set minimo necessario per l'esecuzione, ad esempio CAP_NET_BIND_SERVICE.

Vantaggio: Anche se un processo viene eseguito come root nel contenitore, non ha i permessi di un vero root sull'Host.



SECCOMP-BPF

seccomp-BPF (Secure Computing con Berkeley Packet Filter): È il meccanismo del kernel Linux per **filtrare le chiamate di sistema (syscalls)**.

- **Problema:** Se un processo root compromesso nel contenitore conserva anche solo una piccola capability, potrebbe comunque fare danni utilizzando chiamate di sistema che non dovrebbe usare (ad esempio, chiamate relative al montaggio di filesystem).
- **Soluzione:** Con seccomp-BPF, puoi creare una lista bianca (whitelist) o una lista nera (blacklist) di syscall che un processo può o non può eseguire.



I contenitori di Docker (di default):

- **Applicano** un profilo seccomp predefinito che disabilita circa 44 chiamate di sistema "pericolose".
- *Esempio di syscall disabilitata: mount ()* (per impedire al contenitore di montare o smontare il filesystem dell'Host).

Vantaggio: Fornisce l'ultimo strato di difesa. Anche se un attaccante ha i permessi per eseguire un'azione, il profilo seccomp del kernel può comunque bloccare la chiamata di sistema sottostante.



DOCKER: OPZIONI DI CONFIGURAZIONE

Docker e altri runtime di contenitori sono fondamentalmente un'interfaccia utente amichevole per tutte queste funzionalità del kernel Linux. Esempi di comandi Docker:

Comando	Funzionalità del Kernel Abilitata
<code>docker run (senza flag)</code>	Pivot_root, cgroups, tutti i Namespaces, Capabilities, seccomp
<code>docker run -p 8080:80</code>	Network Namespace (e iptables per la mappatura)
<code>docker run --rm</code>	Pulizia di OverlayFS e cgroups dopo l'uscita
<code>docker run --cap-add=ALL</code>	Capabilities (aggiunge tutte le capabilities, riducendo la sicurezza)
☐ <code>docker run --network=host</code>	Network Namespace (riutilizza il namespace di rete dell'Host, disabilitando l'isolamento di rete)

FAQ



I. COSA CONDIVIDE IL CONTAINER CON L'HOST?

Condivisione Obbligatoria:

- **Il Kernel del Sistema Operativo Host:** I container Linux (come quelli creati da Docker) **condividono lo stesso kernel** dell'host su cui sono eseguiti. Il kernel gestisce tutte le operazioni fondamentali (come l'accesso all'hardware, la gestione della memoria e l'esecuzione dei processi).
- **Risorse Hardware:** La CPU, la memoria RAM e i dispositivi di I/O (Input/Output) sono le stesse risorse fisiche gestite dal kernel. I **cgroups** vengono utilizzati per limitare e allocare queste risorse tra i container in modo controllato.



Condivisione Opzionale:

- **Filesystem Specifici:** Un container può accedere a specifiche directory dell'host (ad esempio, per la persistenza dei dati) tramite i **volumi montati**.
- **Networking:** Per impostazione predefinita, i container vengono eseguiti su una propria rete virtuale, ma le porte possono essere mappate dal container all'interfaccia di rete dell'host (es. "pubblicare" la porta 80 del container sulla porta 8080 dell'host).



2. IL SISTEMA OPERATIVO "DIVERSO"

Kernel (Il Cuore): Il kernel è la parte centrale del sistema operativo che gestisce hardware e risorse. **Questo è sempre condiviso.**

Userland (La Distribuzione): Il "sistema operativo" percepito all'interno del container (ad esempio Ubuntu, Alpine, CentOS) è in realtà solo il **Userland**. Esso è composto da:

1. **Librerie di sistema (es. libc):** Set di funzioni standard utilizzate dai programmi.
2. **Shell e strumenti (es. Bash, ls, apt, yum):** Gli strumenti che permettono di interagire con il sistema operativo.



Spiegazione dell'Indipendenza:

Il meccanismo di **Namespaces** isola queste viste, facendo credere al container di

avere un proprio ambiente isolato:

Namespace	Risorsa isolata (Vista del Container)	Come funziona l'illusione
PID	Processi (ID di processo)	Il container vede solo i propri processi. Il PID 1 del container è il processo principale, ma sul kernel dell'host avrà un PID diverso.
MNT	Filesystem (Gerarchia di directory)	Il container vede il proprio Filesystem (quello definito dall'immagine tarball) e non quello dell'host, a meno che non sia esplicitamente montato.
NET	Interfacce di rete	Il container ha la sua interfaccia di rete virtuale e le sue regole firewall.
UTS	Hostname e dominio	Il container ha un proprio hostname.

Quindi, un container con un **Userland Ubuntu** e uno con un **Userland Alpine** (che hanno set di librerie e strumenti diversi), possono entrambi girare sullo

stesso **Kernel Debian** dell'host, perché il kernel è l'unica cosa che conta per l'esecuzione.

